# クロッキング リソース

**2025.2**

## 概要

この演習では、[Clocking Wizard] を使用してクロック サブシステムを設定し、さまざまなクロック出力や、クロック信号をグローバル クロック ネットワークに接続するためのクロック バッファーを提供する方法を説明します。

この演習では、Verilog と VHDL の両方のデザイン ファイルをサポートしています。ただし、演習の手順および図は Verilog の使用に限定されています。

この演習の所要時間は約 60 分です。

## CloudShare ユーザーのみ

ラボを完了するために 3 回の試行機会が提供されており、各ラボの完了に割り当てられている時間は想定完了時間の 2 倍です。タイマーが一度開始されると、タイマーを一時停止することはできません。ラボを試行するごとに前回の試行がリセットされ、過去の試行での作業内容は保存されません。

## 目的

この演習を完了すると、次のことができるようになります:

- Vivado™ の [IP catalog] から [Clocking Wizard] を使用して、MMCM といくつかの内部バッファーで構成されるクロッキング サブシステムを作成します。
- クロック コアを、追加のクロック バッファーと共に既存のデザインにインスタンス化します
- [Schematic] ビューおよび [Device] ビューを使用して、クロック リージョンならびに配線および分配リソースを調査します。

## はじめに

LogiCORE™ IP Clocking Wizard コアを使用すると、クロック要件に合わせてカスタマイズされたクロック回路の HDL ソース コード ラッパーを簡単に作成できます。このウィザードに


© Copyright 2026 Advanced Micro Devices, Inc.

従ってクロッキング プリミティブに適切な属性を設定でき、ウィザードによって計算されたパラメーターをオーバーライドすることも可能です。Clocking Wizard は、目的のクロック回路を実装するための HDL ラッパーを提供するだけでなく、タイミング ツールによって生成された当該回路のタイミング パラメーター サマリも提供します。

完成したクロック ジェネレーター (生成された clk_core を含む) のアーキテクチャは、下図のような構成になります。

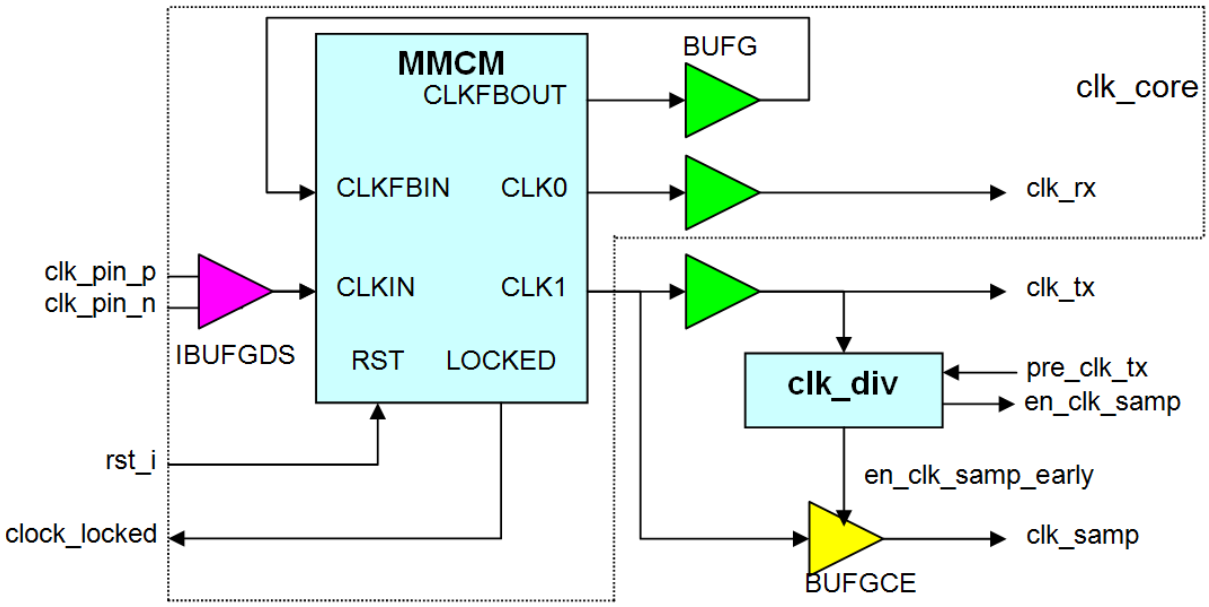



**図 2-1: クロック ジェネレーター (clk_gen) のアーキテクチャ**

### ラボ環境の理解

本コースで提供される演習およびデモは、Linux® プラットフォーム上で動作するように設計されています。

1 つの環境変数が必要です。TRAINING_PATH はラボ ファイルの場所を示します。この変数は CloudShare/CustEd_VM 環境にあらかじめ設定されています。

一部のツールではこの環境変数を直接使用できます (つまり、$TRAINING_PATH が認識されて自動的に展開されます) が、手動での展開が必要なツールもあります (CloudShare/CustEd_VM 環境の場合は /home/amd/training)。各ツールでの対応方法は、ラボの手順書に記載されています。その他の環境では、スクリプトが正しく動作するためにこの変数を定義する必要があります。


© Copyright 2026 Advanced Micro Devices, Inc.
AMD
together we advance_

Vivado™ Design Suite および Vitis™ Unified IDE は、多くのラボで使用される Tcl 環境を提供します。いずれかのツールを起動すると、クリーンな Tcl 環境で起動し、ツールの前回の起動時に残された proc や変数は一切保持されません。

Tcl スクリプトを source した場合や手動で Tcl 変数を設定した後にツールを閉じた場合、ツールを再度開いたときに Tcl スクリプトを再度 source し、演習に必要な変数を設定し直す必要があります。これはターミナル ウィンドウにも当てはまり、新しいターミナルを開くとすべての変数設定がクリアされます。

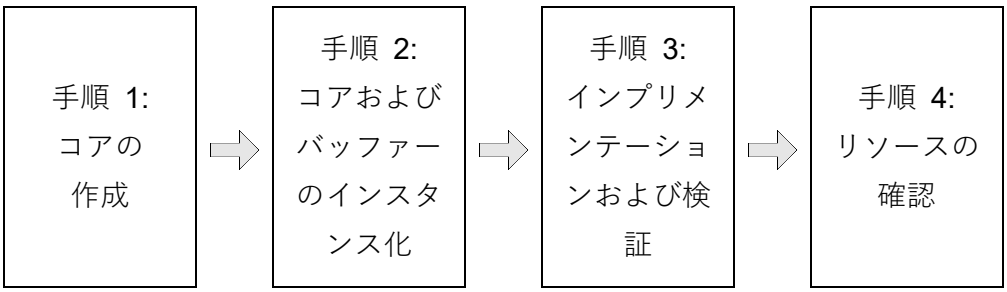
**表記規則**

各引数の使用方法を説明するために、正式な表記法が使用されています。以下は、よく使用される記号の一部です。

| シンボル | 説明 | 例 | 説明 |
|---|---|---|---|
| **<text>** | フィールドを示します | `cd <dir>` | `<dir>` はディレクトリ名を表します。`<` および `>` 記号は入力しません。移動先のディレクトリが `XYZ` の場合、環境に `cd XYZ` と入力します。 |
| **[text]** | オプションの引数を示します | `ls [ \| more]` | これは ls <Enter> または ls \| more <Enter> と解釈できます。最初の例は現在の Linux ディレクトリ内のファイルを一覧表示し、2 番目の例は現在の Linux ディレクトリ内のファイルを一覧表示しますが、さらに出力を more ツールに渡してページ単位で表示します。ここで、パイプ記号 (\|) は Linux の演算子です。 |
| **\|** | 選択肢を示します | `cmd <ZCU104 \| VCK190>` | cmd コマンドは 1 つの引数を取ります。この引数は ZCU104 または VCK190 のいずれかです。cmd ZCU104 または cmd VCK190 のいずれかを入力します。 |


© Copyright 2026 Advanced Micro Devices, Inc.

## 全体の流れ




## clk_core コアの作成 手順 1

作業ディレクトリから提供されている未完成の演習デザインを開いて演習を開始します。次に、デザイン要件に従ってクロック IP コア（[IP Catalog] から利用可能）をカスタマイズします。

Vivado Design Suite を開くには、次の 2 つの方法があります。

### 1-1. [Vivado Design Suite] を開きます。

**1-1-1.** **[taskbar]** から **[Vivado]** アイコン **()** をクリックします。

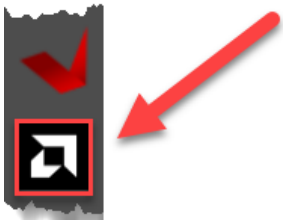


**図 2-2: Taskbar からの Vivado Design Suite の起動**

**注: 起動には少し時間がかかります。使用環境によってアイコンの並び順が異なる場合があります。**

または、Linux ターミナル ウィンドウ (<Ctrl + Alt + T>) を開き、次のように入力します。

```
source /opt/amd/2025.2/Vivado/settings64.sh; vivado
```


© Copyright 2026 Advanced Micro Devices, Inc.
AMD together we advance_

**注記: このインストール パスは、CustEd VM および CloudShare 環境で有効です。ご使用の環境に適したパスを使用してください。**

ツールを起動すると、[Welcome] ウィンドウが表示されます。ここから、新規プロジェクトの作成、既存プロジェクトを開く操作、Tcl コマンドの入力、ドキュメントやサンプルへのアクセスが可能です。

**1-1-2.** **[オプション]** 表示される情報が多いため、ウィンドウを最大化します。

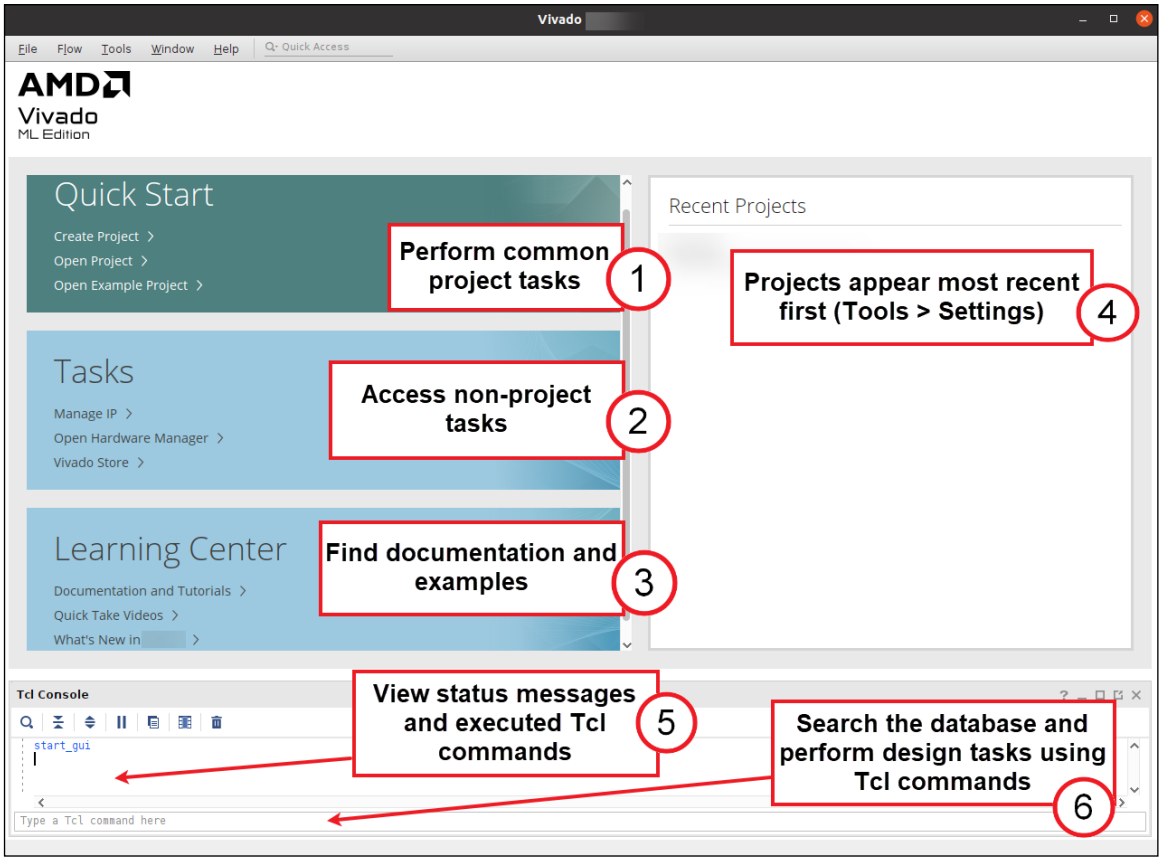



**図 2-3: Vivado Design Suite Welcome Screen**

**ヒント: [Tcl Console] が表示されていない場合は、[Tcl Console] タブをダブルクリックして表示させてください。**

## 1-2. [Tcl Console] を使用してプロジェクトを解凍します。

**1-2-1.** **[Tcl Console]** に次のコマンドを入力して、プロジェクトを解凍します:

```
exec unzip $::env(TRAINING_PATH)/clk_resources/lab/<KCU105 |
SCU035>/verilog.zip -d
$::env(TRAINING_PATH)/clk_resources/lab/<KCU105 |
SCU035>/verilog
```


© Copyright 2026 Advanced Micro Devices, Inc.

## 1-3. 既存の Vivado Design Suite プロジェクト wave_gen.xpr を開きます。

**1-3-1.** **[Quick Start]** セクションから **[Open Project]** をクリックします **(1)**。

[Open Project] ダイアログ ボックスが開きます (2)。

**1-3-2.** **[Look in]** フィールド **(3)** で次のディレクトリを参照します。

`$TRAINING_PATH/clk_resources/lab/<KCU105 | SCU35>/verilog`

**注記: [drop-down arrow] はディレクトリ階層を示します。**

**1-3-3.** 表示されたファイルから **wave_gen.xpr** を選択します **(4)**。

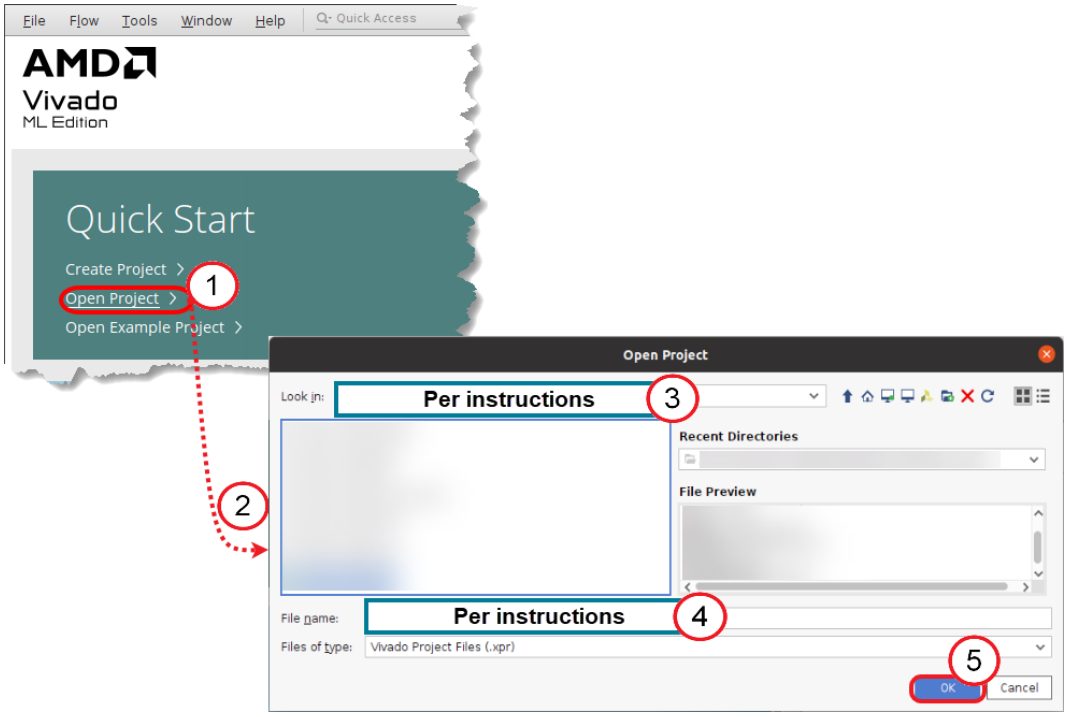



**図 2-4: 既存のプロジェクトを開く**

**1-3-4.** **[OK]** をクリックして、選択したプロジェクト **(5)** を開きます。

これで、プロジェクトが Vivado Design Suite で開きます。

## 1-4. Vivado IDE の [IP catalog] から [Clocking Wizard] を起動します。

**1-4-1.** **[Flow Navigator]** の **[Project Manager]** にある **[IP Catalog]** をクリックします。

または、[Window] > [IP Catalog] を選択することもできます。

[IP Catalog] ウィンドウがメイン ワークスペース エリアに開きます。

**1-4-2.** **[IP catalog]** で **[Foundational Elements] > [Clock and Reset]** を展開します。


© Copyright 2026 Advanced Micro Devices, Inc.

**1-4-3.** **[Clocking Wizard]** をダブルクリックして、選択した **IP** をカスタマイズします。

**注: IP 名全体を表示するには、[Name] 列の幅を広げてください。**

または、右クリックして [Customize IP] を選択することもできます。

[Customize IP] ウィンドウが開きます。

### 質問 1

なぜ [Clocking Wizard] を使用してコアを作成するのでしょうか。つまり、なぜ単に MMCM プリミティブをインスタンス化しないのでしょうか。

---

---

---

## 1-5. MMCM クロック コアを使用し、名前を clk_core に設定します。入力クロック周波数を 300 MHz に設定します。

**1-5-1.** **[Component Name]** フィールドに **clk_core** と入力します。

**1-5-2.** **[Clocking Options]** タブを選択します。

**1-5-3.** **[Primitive]** セクションで **[MMCM]** が選択されていることを確認します。

**1-5-4.** **[Clocking Features]** セクションで、**[Frequency Synthesis]** および **[Phase Alignment]** オプションが選択されていることを確認します。

**1-5-5.** **[Jitter Optimization]** セクションで **[Balanced]** が選択されていることを確認します。

**1-5-6.** **[Primary Input Clock]** の **[Input Frequency]** フィールドを **300.000 MHz** に設定します。

**1-5-7.** **[Primary Input Clock]** の **[Source]** フィールドを **[Differential clock capable pin]** に設定します。

**注: コアに対して指定された [Primary Input Clock Frequency] オプションは、ウィザードが以降のステージで指定される出力を生成するための適切なモードおよびパラ**


© Copyright 2026 Advanced Micro Devices, Inc.

**メーターを計算するために使用されます。この周波数がボード上のクロック ソースの周波数と一致しない場合、出力は要求された出力周波数と一致しなくなります。**

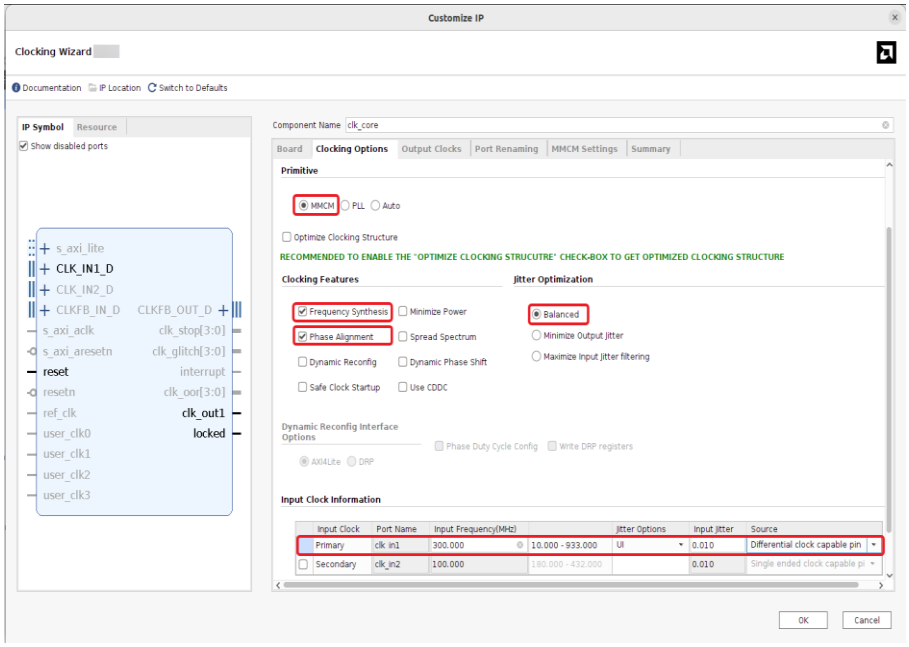



**図 2-5: Clocking Options タブ**

## 1-6. MMCM モジュールから 2 つの出力クロック clk_out1 および clk_out2 を生成します。出力周波数は clk_out1 を 200 MHz、clk_out2 を 193.75 MHz に設定します。

**1-6-1.** **[Clocking Wizard]** で **[Output Clocks]** タブを選択します。

**1-6-2.** **clk_out1** 出力クロックの **[Output Freq (MHz) Requested]** フィールドに値として **200** を入力します。

**1-6-3.** チェック ボックスを使用して **clk_out2** 出力クロックを有効にします。

**1-6-4.** **clk_out2** 出力クロックの **[Output Freq (MHz) Requested]** フィールドに値として **193.75** を入力します。

**1-6-5.** 両方の出力クロックで **[Phase]** フィールドが **0.000** に設定されていることを確認します。

**1-6-6.** 両方の出力クロックで **[Duty Cycle]** フィールドが **50%** に設定されていることを確認します。

**1-6-7.** **clk_out1** の **[Drives]** フィールドが **BUFG** に設定されていることを確認します。

**1-6-8.** **clk_out2** の **[Drives]** フィールドを **[No buffer]** に変更します。

バッファーは、後の手順で手動でインスタンス化されます。


© Copyright 2026 Advanced Micro Devices, Inc.

**1-6-9.** **[Clocking Feedback Source]** セクションで **[Automatic Control On-Chip]** オプションが選択されていることを確認します。

**1-6-10.** **[Enable Optional Inputs / Outputs for MMCM/PLL]** セクションで、**[reset]** および **[locked]** オプションのみが選択されていることを確認します。

**1-6-11.** **[Reset Type]** セクションで **[Active High]** オプションが選択されていることを確認します。

**1-6-12.** **[Phase Shift Mode]** で **[Waveform]** オプションが選択されていることを確認します。

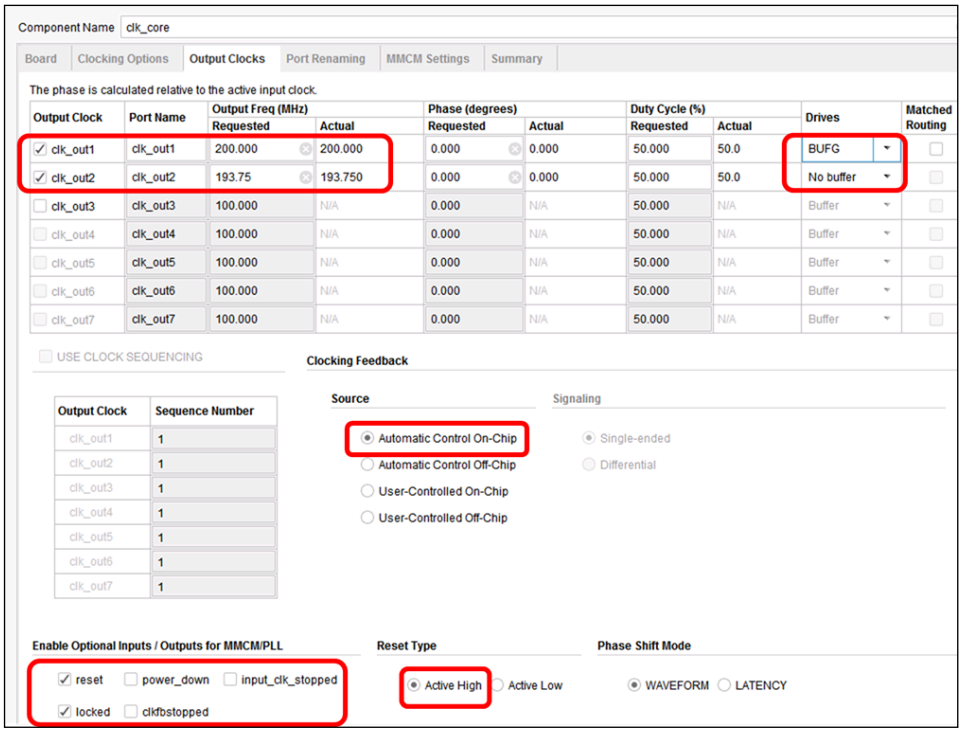



**図 2-6: Output Clocks タブ**

## 質問 2

clk_out2 に [No buffer] オプションを使用するのはなぜですか?


© Copyright 2026 Advanced Micro Devices, Inc.

## 1-7. [MMCM Settings] タブのデータを使用して、以下の質問に答えてください。

### 1-7-1. [Clocking Wizard] で [MMCM Settings] タブを選択し、設定を確認します。

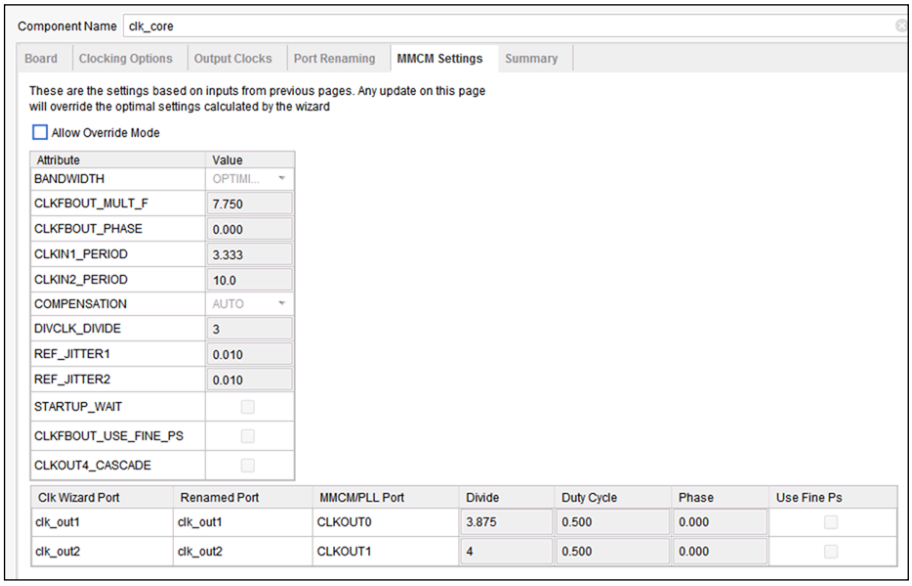



**図 2-7: MMCM Settings タブ - KCU105**

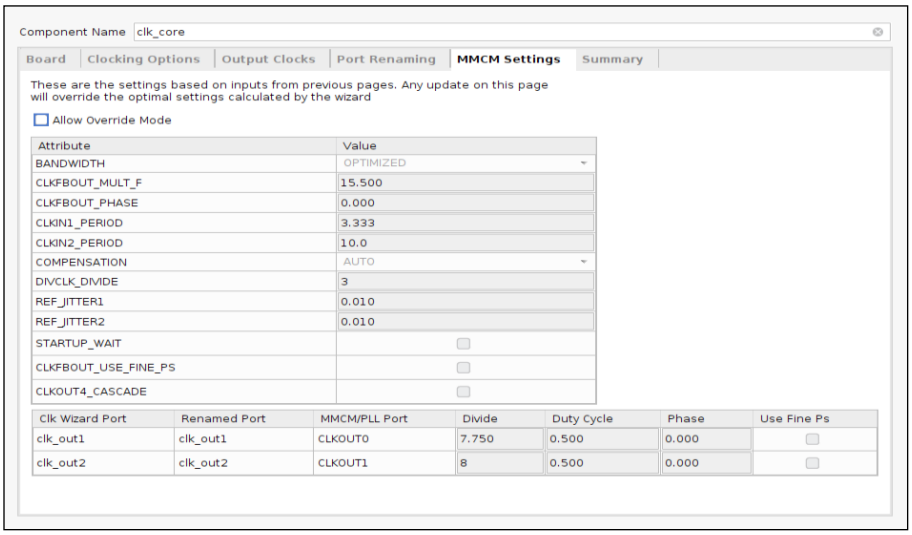



**図 2-8: MMCM Settings タブ - SCU035**

### 質問 3

[Clocking Wizard] に表示されている入力クロック周波数、および DIVCLK_DIVIDE と CLKFBOUT_MULT_F パラメーターの値に基づくと、[Clocking Wizard] は VCO 周波数として何を選択していますか?


© Copyright 2026 Advanced Micro Devices, Inc.
AMD
together we advance_

### 質問 4

[Clocking Wizard] に表示されている VCO 周波数と 2 つのクロックの [Divide] の値から、出力クロックは要求された周波数と一致しますか?

## 1-8. [Summary] タブを確認します。

### 1-8-1. [Summary] タブを選択します。

### 1-8-2. [Primary Input Clock Attributes] セクションを確認します。

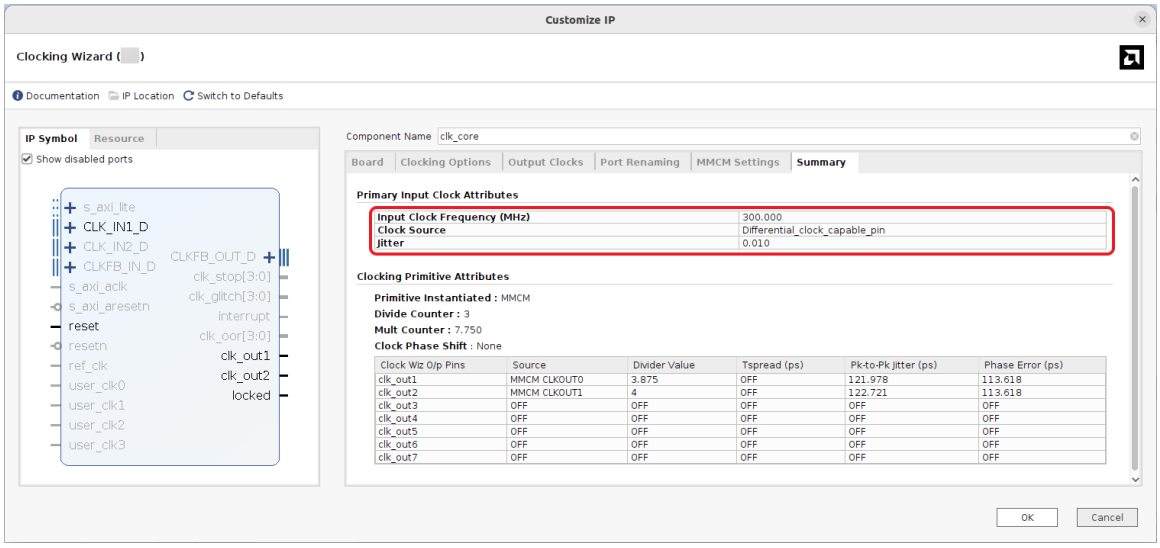



**図 2-9: Clocking Wizard Summary - KCU105**


© Copyright 2026 Advanced Micro Devices, Inc.

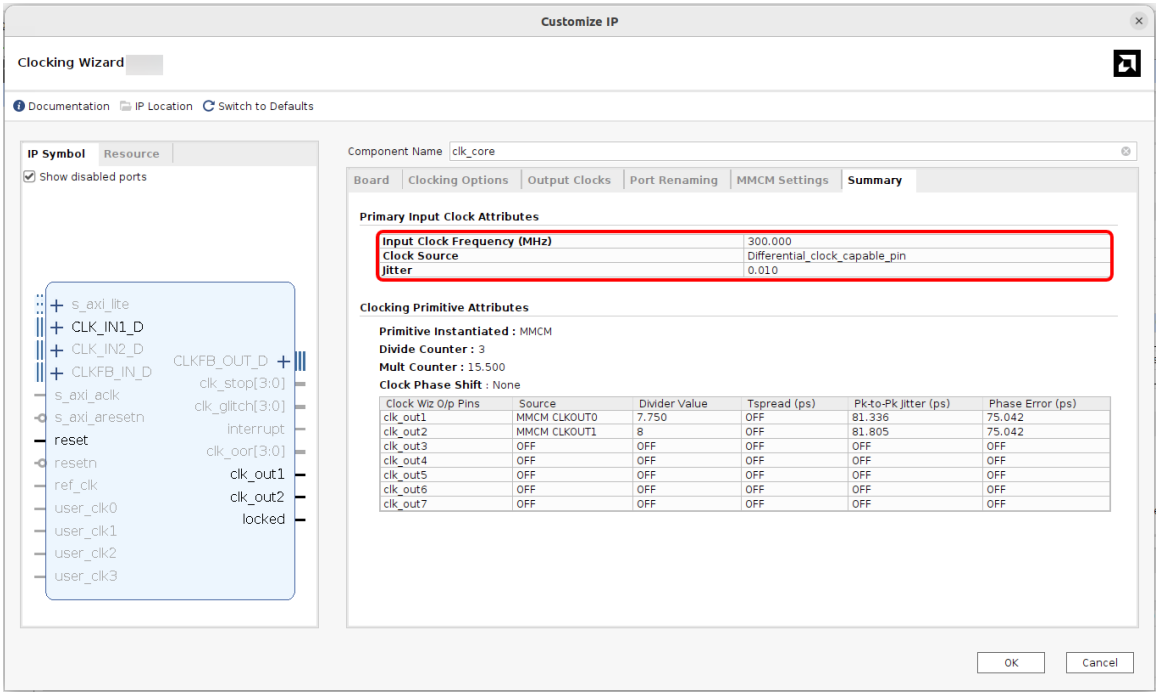



**図 2-10: Clocking Wizard Summary - SCU035**

## 1-9. コアおよび出力プロダクトを生成します。

**1-9-1.** **[Customize IP]** ダイアログ ボックスで **[OK]** をクリックします。

[Generate Output Products] ダイアログ ボックスが開き、生成される出力プロダクトの一覧が表示されます。

**1-9-2.** ダイアログボックスで、デフォルトの **[Synthesis Options]** のまま **[Generate]** をクリックします。

**1-9-3.** **[Generate Output Products]** ダイアログ ボックスが表示された場合は、**[OK]** をクリックします。

新しいコアはバックグラウンドで論理合成されます。[Design Runs] ビューのステータスを確認することで、実行中であることを確認できます。論理合成が完了すると、右下に通知が表示されます。

[Sources] > [Hierarchy] ビューに新しいファイル clk_core.xci が追加されていることを確認してください。このソース ファイルは、コンポーネントがいずれかの HDL ソース ファイルにインスタンシエートされるまで、デザイン階層には含まれません。


© Copyright 2026 Advanced Micro Devices, Inc.
AMD together we advance_

## 1-10. clk_core の HDL 機能モデルを表示し、[IP Catalog] から実装されたコアが期待どおりのアーキテクチャになっていることを確認します。

**1-10-1.** **[Sources]** ビューで **[IP Sources]** タブを選択します。

**1-10-2.** **[IP] > [clk_core] > [Synthesis]** を展開します。

**1-10-3.** **[clk_core_clk_wiz.v]** をダブルクリックして、**HDL** 機能モデルを表示します。

**1-10-4.** **[Clocking Wizard]** によって生成されたソース コードを確認します。

**KCU105 の場合:**

このファイルには、次のインスタンシエートされたコンポーネントが含まれています: 1 つの MMCME3_ADV、1 つの IBUFDS、および 2 つの BUFG。

差動入力クロック clk_in1_p および clk_in1_n は IBUFDS を駆動し、これがさらに MMCME3_ADV の CLKIN1 ポートを駆動します。

MMCME3_ADV の CLKOUT0 出力は BUFG を駆動し、これがモジュールの clk_out1 出力ポートに接続されます。

MMCME3_ADV の CLKOUT1 出力は、モジュールの clk_out2 ポートを直接 (バッファーなしで) 駆動します。

MMCME3_ADV の CLKFBOUT 出力は、BUFG を介して MMCME3_ADV の CLKFBIN 入力を駆動します。

Verilog では、MMCM 属性はインライン パラメーター割り当てによって渡されます。

**SCU035 の場合:**

このファイルには、次のインスタンス化されたコンポーネントが含まれています: MMCME4_ADV、IBUFDS、および 2 つの BUFG。

差動入力クロック clk_in1_p および clk_in1_n は IBUFDS を駆動し、これがさらに MMCME4_ADV の CLKIN1 ポートを駆動します。

MMCME4_ADV の CLKOUT0 出力は BUFG を駆動し、BUFG はモジュールの clk_out1 出力ポートに接続されます。

MMCME4_ADV の CLKOUT1 出力は、モジュールの clk_out2 ポートを直接 (バッファーなしで) 駆動します。

MMCME4_ADV の CLKFBOUT 出力は、BUFG を介して MMCME4_ADV の CLKFBIN 入力を駆動します。


© Copyright 2026 Advanced Micro Devices, Inc.

Verilog では、MMCM 属性はインライン パラメーター割り当てを介して渡されます。

**質問 5**

コアが既に作成されている場合、デザインに既に接続されているでしょうか。なぜこのデザインにクロッキング コアをインスタンス化する必要があるのでしょうか。

---

---

---

**1-10-5.** **HDL** ソースの確認が終了したら、タブ名の横にある **[X]** をクリックして **[clk_core_clk_wiz.v]** タブを閉じます。

## clk_core コアおよびバッファーのインスタンス化 手順 2

### 2-1. インスタンス化テンプレートのテキストをコピーして、clk_gen_us_clk_resources.v ファイル内のコメント「Instantiate clk_core – generated by the Clocking Wizard」の後に貼り付け、clk_core コンポーネントをインスタンス化します。

**2-1-1.** **[Sources]** ビューの **[IP Sources]** タブを選択します。

**2-1-2.** **[IP] > [clk_core] > [Instantiation Template]** を展開します。

**2-1-3.** **clk_core.veo** を選択してダブルクリックし、テキスト エディターでインスタンス化テンプレートを開きます。

テンプレートが提供される言語は、プロジェクトのビルド時に選択した言語によって設定されています。これはいつでも変更できます。

**2-1-4.** **[clk_core.veo]** ウィンドウで、「**Begin Cut here for INSTANTIATION Template**」と「**End INSTANTIATION Template**」のコメント行の間にあるすべてのテキストを選択し、右クリックして **[Copy]** を選択することで、**clk_core** コアのインスタンス化テンプレートをコピーします。

**2-1-5.** **[Sources]** ビューで **[Hierarchy]** タブを選択します。

**2-1-6.** **wave_gen** の横にある **[>]** をクリックして、**wave_gen** 階層を展開します。


© Copyright 2026 Advanced Micro Devices, Inc.

**2-1-7.** **clk_gen_us_clk_resources.v** をダブルクリックして、**[Text Editor]** でソース コードを開きます。

**2-1-8.** 右クリックして **[Paste]** を選択し、コピーしたテキストを **clk_gen_us_clk_resources.v** ファイル内のコメント「**Instantiate clk_core – generated by the Clocking Wizard**」の後に貼り付けます。

## 2-2. インスタンス名を変更し、ポート接続を入力してインスタンス化を完了します。

**2-2-1.** インスタンス名を変更し、次のようにポート接続を入力してインスタンシエーションを完了します。

**[Verilog ユーザー]:**

```
clk_core clk_core_i0 (
// クロック出力ポート
.clk_out1(clk_rx),                // output clk_out1
    .clk_out2(clk_tx_unbuf),      // output clk_out2
// ステータスおよび制御信号
.reset(rst_i),                    // 入力リセット
    .locked(clock_locked),        // 出力ロック
// クロック入力ポート
.clk_in1_p(clk_pin_p),            // input clk_in1_p
    .clk_in1_n(clk_pin_n));       // input clk_in1_n
```

### 質問 6

[Vivado IDE] の [Verilog editor] では、ポートが有効なネットに接続されているかどうかはどのように示されますか。


© Copyright 2026 Advanced Micro Devices, Inc.

## 2-3. BUFG の [Language Templates] からテキストをコピーし、clk_gen_us_clk_resources.v ファイル内のコメント「Instantiate BUFG here」の後に貼り付けて、インスタンス化を完了します。

**2-3-1.** **[Tools] > [Language Templates] ()** ウィンドウを開きます。

**2-3-2.** **[Verilog] > [Device Primitive Instantiation] > <Kintex Ultrascale | Spartan UltraScale+> [CLOCK] > [BUFFER]** を展開します。

**2-3-3.** **[General Clock Buffer (BUFG)]** を選択します。

**2-3-4.** **[Preview]** ウィンドウの **[Language Template]** からインスタンス化テンプレート全体を選択します。

**2-3-5.** 右クリックして **[Copy]** を選択するか、**<Ctrl + C>** を押してテンプレートをコピーします。

**2-3-6.** **[main window]** で **[clk_gen_us_clk_resources.v]** タブを選択して、**clk_gen** ファイルに戻ります。

**2-3-7.** **BUFG** インスタンス化テンプレートの行を **Instantiate BUFG here** コメントの後に貼り付けます。

**2-3-8.** インスタンス名とポート接続を次のように変更して、インスタンス化を完了します。

**[Verilog ユーザー]:**

```
BUFG BUFG_clk_tx_i0 (
        .O(clk_tx), // 1 ビット出力: バッファー
        .I(clk_tx_unbuf)   // 1 ビット入力: バッファー
    );
```

## 2-4. BUFGCE の言語テンプレートからテキストをコピーし、clk_gen_us_clk_resources.v ファイル内のコメント Insert BUFGFE Here の後に貼り付けて、インスタンス化を完了します。

**2-4-1.** 上記の手順を繰り返して、**clk_samp** 用の **BUFGCE** をインスタンス化します。テンプレート **[General Clock Buffer with Clock Enable (BUFGCE)]** を使用します。

**2-4-2.** インスタンス名とポート接続を次のように変更して、インスタンス化を完了します。

**[Verilog ユーザー]:**

```
BUFGCE #(
.CE_TYPE("SYNC"),                // ASYNC, HARDSYNC, SYNC
.IS_CE_INVERTED(1'b0),           // CE のプログラマブル反転
.IS_I_INVERTED(1'b0),            // I のプログラマブル反転
```


© Copyright 2026 Advanced Micro Devices, Inc.

```
.SIM_DEVICE("ULTRASCALE")   // ULTRASCALE
)
BUFGCE_clk_samp_i0 (
.O(clk_samp),     // 1 ビット出力: バッファー
.CE(en_clk_samp_early), // 1 ビット入力: バッファー イネーブル
.I(clk_tx_unbuf)       // 1-bit input: Buffer
);
```

**2-4-3.** **[X]** をクリックして **[Templates]** ウィンドウを閉じます。

**2-4-4.** **[File] > [Text Editor] > [Save File]** を選択して **clk_gen_us_clk_resources.v** を保存するか、メイン ビューの **[clk_gen.v]** タブにある アイコンをクリックします。


© Copyright 2026 Advanced Micro Devices, Inc.

## デザインの実装と検証 手順 3

次に、clk_core IP が clk_gen.v モジュール内で適切にインスタンシエートされていることを確認し、デザインをインプリメントして、[Clocking Wizard] によってどのようなクロック リソースが使用されているかを確認します。

### 3-1. clk_core が clk_gen.v/vhd モジュール内で正しくインスタンス化されていることを確認します。

**3-1-1.** **[Hierarchy]** ウィンドウを選択します。

**3-1-2.** **clk_core.xci** ファイルが正しく配置されていることを確認します。

clk_core の正しい位置を確認するには、clk_gen_i0 を展開する必要があります。

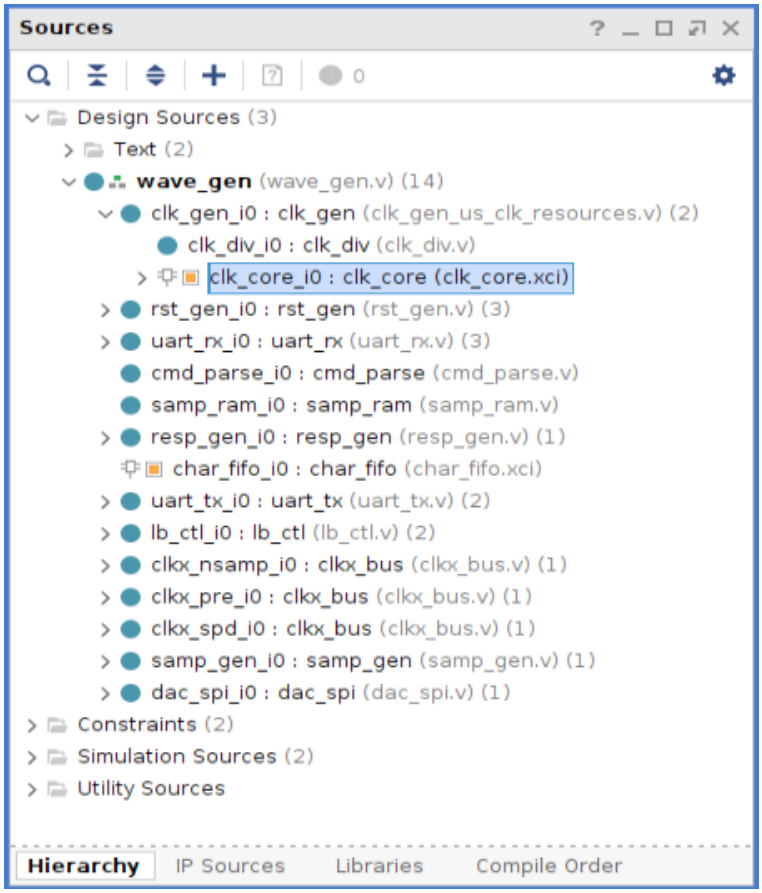



**図 2-11: clk_core が正しくインスタンス化された Hierarchy ウィンドウ**


© Copyright 2026 Advanced Micro Devices, Inc.
AMD together we advance_

## 3-2. デザインをインプリメントします。

**3-2-1.** **[Flow Navigator]** の **[Implementation]** の下にある **[Run Implementation]** をクリックします **(1)**。

または、[Flow] > [Run Implementation] (2) を選択することもできます。

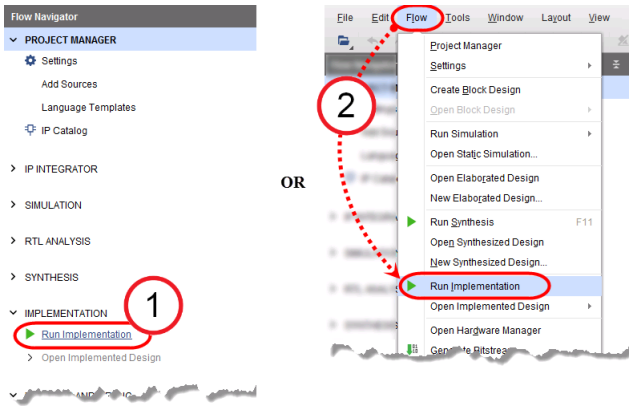



**図 2-12: Run Implementation の選択**

**注記: プロンプトが表示された場合は、必要に応じて [Yes] をクリックして、先に論理合成を実行してください。**

ツールによってデザインのインプリメントに必要なすべてのプロセスが実行される点に留意してください。これは、合成済みネットリストがまだ存在しない場合、Vivado Design Suite がインプリメンテーションを実行する前に論理合成を実行することを意味します。

インプリメンテーションが完了すると、[Implementation Completed] ダイアログ ボックスが開きます。このダイアログ ボックスでは、インプリメント済みデザインを開く、ビットストリームを生成する、またはレポートを表示することを選択できます。

**3-2-2.** 再度 **[OK]** をクリックして、選択した **run** を起動します。

**3-2-3.** **[Open Implemented Design]** を選択します。

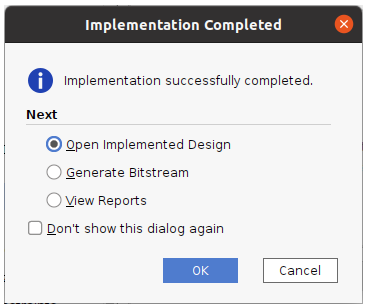



**図 2-13: Implementation Completed ダイアログ ボックス**


© Copyright 2026 Advanced Micro Devices, Inc.

**3-2-4.** **[OK]** をクリックします。

**3-2-5.** クリティカル警告が表示された場合は、ダイアログ ボックスの **[OK]** をクリックします。

**注: この処理が完了するまでに約 5 ～ 7 分かかる場合があります。**

## 3-3. インプリメント済みデザインの [Utilization] レポートを生成します。

**3-3-1.** **[Flow Navigator]** の **[Open Implemented Design]** の下にある **[Report Utilization]** をクリックします。

または、[Reports] > [Report Utilization] を選択することもできます。

[Report Utilization] ダイアログ ボックスが開きます。

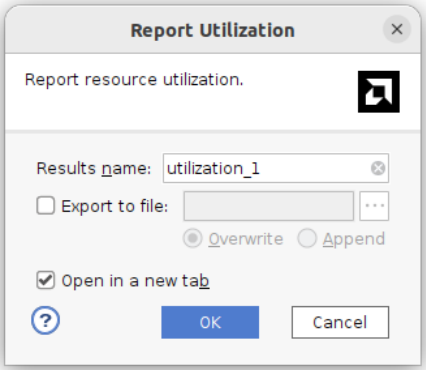



**図 2-14: Report Utilization ダイアログ ボックス**

**3-3-2.** デフォルト設定のまま **[OK]** をクリックします。

[Design Utilization Summary] ウィンドウが下部の [Utilization] タブに開きます。

デバイス リソースは、各 RTL デザイン モジュールごとに階層形式でリストされることに注意してください。

[Utilization] レポートが Vivado IDE の下部に表示されます。

**3-3-3.** **[Utilization]** レポートを調べ、使用されているクロック リソースの数を確認します。

**3-3-4.** **[Utilization]** レポートの **[Clocking]** セクションで、さまざまなオブジェクト タイプをクリックします。

### 質問 7

完成したデザインでは、いくつの MMCM およびグローバル クロック バッファー リソースが使用されましたか？


© Copyright 2026 Advanced Micro Devices, Inc.
AMD
together we advance_

### 3-4. [Schematic] ビューアーを使用してクロック ジェネレーターの実装を確認し、次の質問に答えてください。

**3-4-1.** **[Netlist]** ウィンドウでインスタンス **clk_gen_i0 (clk_gen)** を選択します。

**3-4-2.** 右クリックし、**[Schematic]** を選択します。

**3-4-3.** **clk_gen_i0** 内の **[+]** アイコンをクリックして展開します。

**3-4-4.** **clk_core_i0** 内の **[+]** アイコンをクリックして展開します。

**3-4-5.** 次の手順のため、**[Schematic]** ビューを開いたままにしておきます。

## クロック リソースの確認 手順 4

ここでは、[Schematic] および [Device] ビューアーを使用して、デザインで使用されているクロック リソースを視覚化します。

### 4-1. [Schematic] ビューおよび [Device] ビューを使用して、MMCM と 4 つのクロック バッファーの場所を特定します。

**4-1-1.** **KCU105 の場合: [Schematic] タブで MMCME3_ADV インスタンスを選択します。**

**SCU035 の場合: [Schematic] タブで MMCME4_ADV インスタンスを選択します。**

**4-1-2.** メイン ウィンドウで **[Device]** タブを選択して、**[Device]** ビューに切り替えます。

**4-1-3.** **[Device]** ビューの水平ツールバーで **[Routing Resources]** アイコン **()** が選択されていることを確認します。

**4-1-4.** **[Show Cell Connections]** アイコン **()** をクリックし、**[Hide All]** を選択します。

これにより、選択したセルの接続性が有向 “rats-nest” 表示で示されますが、これは後ほど行います。

**4-1-5.** **[Device]** ビューの水平ツールバーで、**[Auto Fit Selection]** アイコン **()** が選択されていないことを確認します。

**4-1-6.** **[Fit Selection]** アイコン **()** をクリックするか、**<F9>** を押して、選択した **MMCME3_ADV/MMCEME4_ADV** セルを拡大表示します。


© Copyright 2026 Advanced Micro Devices, Inc.

**4-1-7.** クロック領域が表示されるまでズームアウトし、次の質問に答えてください。

**4-1-8.** **[Schematic]** ビューに戻り、以下の各インスタンスに対して上記の手順を繰り返します。

- clkout1_buf (clk_rx 用の BUFG)
- BUFG_clk_tx_i0 (clk_tx 用の BUFG)
- BUFGCE_clk_samp_i0 (clk_samp 用の BUFGCE)
- clkf_buf (クロック フィードバック用 BUFG)
- 各バッファーを右クリックして [Mark] を選択すると、識別しやすいようにすべてのビュー ([Device]、[Netlist]、および [Schematic] ビュー) でこれらのリソースがマークされます。

### 質問 8

MMCM と 4 つのクロック バッファーはどのクロック領域に配置されていますか?

---

---

---

## 4-2. [Schematic] ビューアーを使用して、clk_tx ネットをハイライトします。

**4-2-1.** **[Schematic]** ビューに戻ります。

**4-2-2.** **clk_gen_i0** セルの **clk_tx** ピン **(BUFGCE** の出力**)** に接続されている **clk_gen_i0** 内のネットを選択します。

ピンではなく、必ずネットを選択してください。


© Copyright 2026 Advanced Micro Devices, Inc.

**4-2-3.** **[Schematic]** ウィンドウで右クリックし、**[Highlight]** から識別しやすい色を選択して、選択したネットをハイライト表示します。

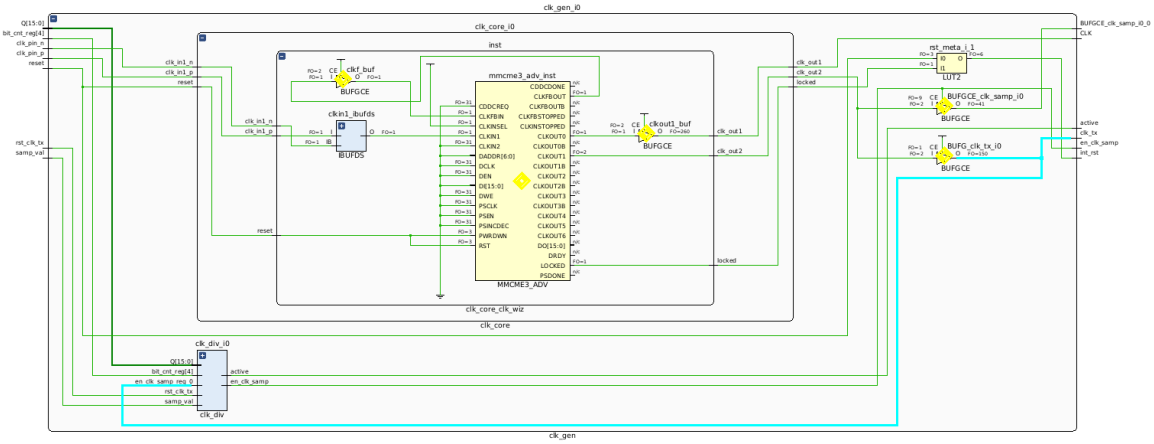



**図 2-15: Schematic でハイライトされた clk_tx ネット (Verilog デザイン) - KCU105**

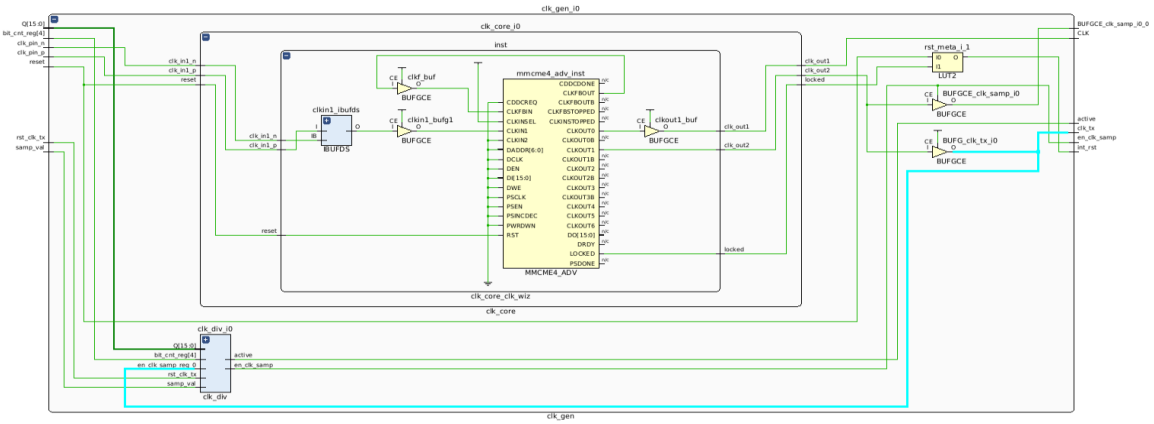



**図 2-16: Schematic でハイライトされた clk_tx ネット (Verilog デザイン) - SCU035**

## 4-3. [Device] ビューアーを使用してクロック構造の確認を開始します。

**4-3-1.** **[Device]** ビューに戻り、ズームアウトしてクロック領域を表示します。

### 質問 9

clk_tx ドメインはどのクロック リージョンで使用されていますか?


© Copyright 2026 Advanced Micro Devices, Inc.

**4-4. [Device viewer] を使用して、各クロック リージョンで clk_tx ドメインによってどのクロック ルーティング トラックおよびディストリビューション トラックが使用されているかを確認します。当該ドメインを駆動する BUFGCE から開始し、リージョン X2Y2 にあるスイッチ ボックスに進んで、[Device viewer] で接続を解析します。**

**4-4-1.** **[Schematic]** ビューに戻り、**clk_tx** ドメインを駆動する **BUFGCE** を選択します。

**4-4-2.** **[Device]** ビューに戻り、**[Fit Selection]** アイコン **()** を使用するか、**<F9>** を押して **BUFGCE** を拡大表示します。

**4-4-3.** 未使用のトラック **(**灰色で表示されているトラック**)** にカーソルを合わせます。

ポップアップにトラックに関する情報が表示されている点に注目してください。

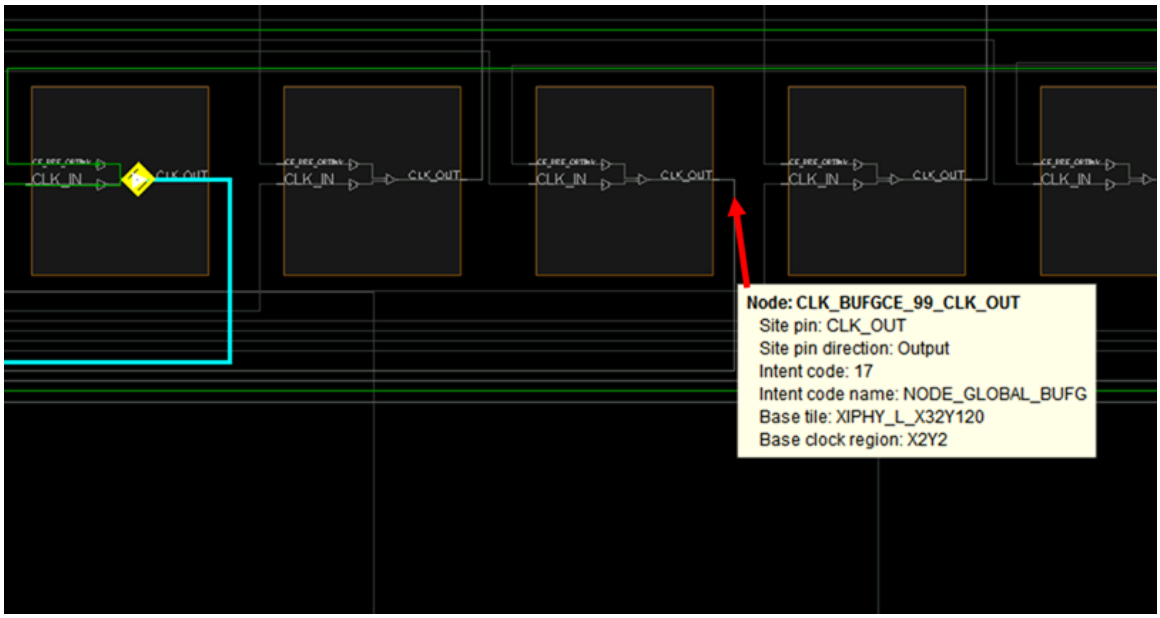



**図 2-17: Device View の未使用トラック**

ネットで使用されているトラック (ハイライト表示された clk_tx ネットなど) にカーソルを合わせると、ポップアップにそのネットに関する情報が表示されます。

トラックに関する情報を確認するため、[Device viewer] ですべてのネットの表示をオフにします。

**4-4-4.** **[Show Cell Connections]** アイコン **()** をクリックします。

**4-4-5.** **[Show Cell Connections]** ダイアログ ボックスで **[Show All]** をクリックして、現在選択されているセル **(clk_tx** を駆動する **BUFGCE)** の接続性 **(**ラッツネスト**)** ビューを表示します。

確認後、[Hide All] を選択して接続表示を非表示にします。


© Copyright 2026 Advanced Micro Devices, Inc.
AMD
together we advance_

**4-4-6.** ハイライトされたネットが **T** 字交差を形成しているように見える箇所が確認できるまでズームアウトします。

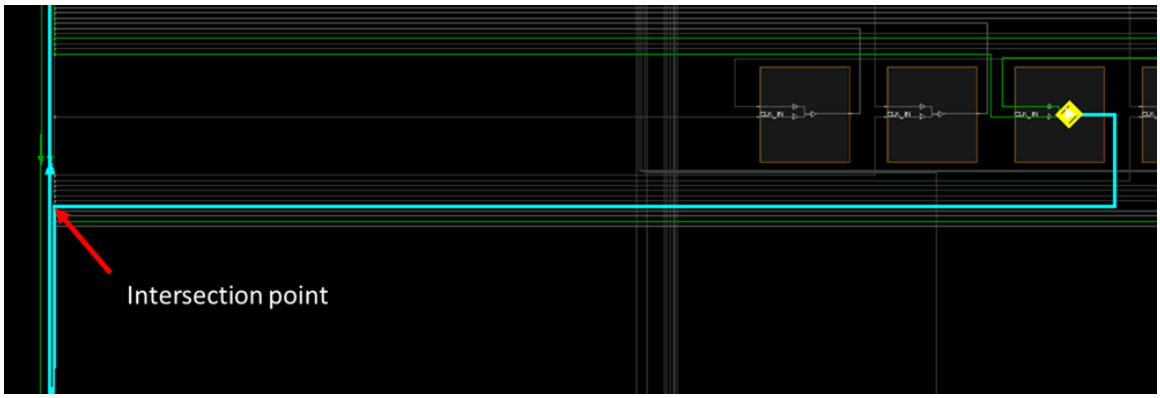



**図 2-18: clk_tx ネットの T 分岐点**


© Copyright 2026 Advanced Micro Devices, Inc.

**4-4-7.** 交点を拡大します。

十分に拡大すると、BUFG からの信号は実際には交差しておらず、ファブリック スイッチング ボックスに入力されていることが確認できます。

**注記: スイッチ ボックスは非常に大きく、ほぼクロック領域全体の高さがあります。**

**ヒント: ナビゲーション ジェスチャを使用してズームを制御します:**

- ズーム アウト: マウスの左ボタンを押したまま右上へ動かして離します。
- ズームイン: マウスの左ボタンを押したまま、左下へ移動して離します。
- [Zoom box]: マウスの左ボタンを押したまま右下に移動し、離します。
- [Zoom full]: マウスの左ボタンを押したまま右上に動かし、離します。

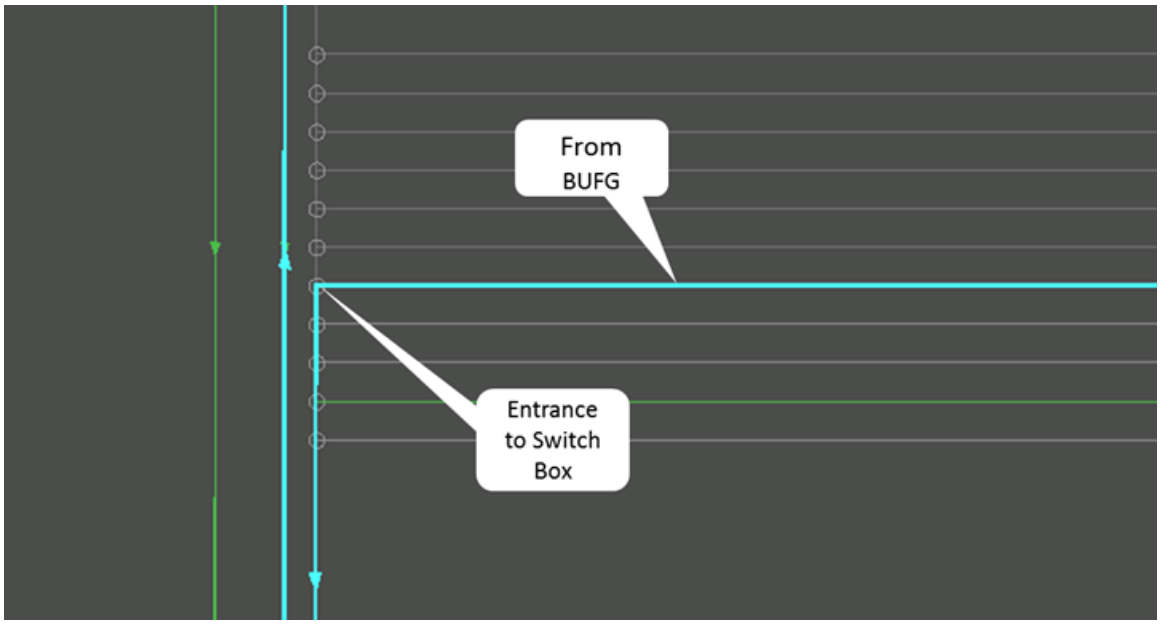



**図 2-19: スイッチ ボックスに入力されるバッファされた clk_tx**

スイッチ ボックス自体の内部で clk_tx ネットをトレースすることは非常に困難である点に注意してください。

**4-4-8.** **clk_tx** ネットがスイッチ ボックスから出ているすべての箇所を拡大表示します。

**4-4-9.** ズームアウトおよびパンして、ボックスを表示します。

clk_tx ネットをハイライトした状態でズームアウトすると、スイッチ ボックス全体が clk_tx ネットで埋め尽くされているように見えます。


© Copyright 2026 Advanced Micro Devices, Inc.

**4-4-10.** **[Device]** ビューで、クロック リージョン **X2Y2** および **X2Y1** が表示されるまでズームアウトします。

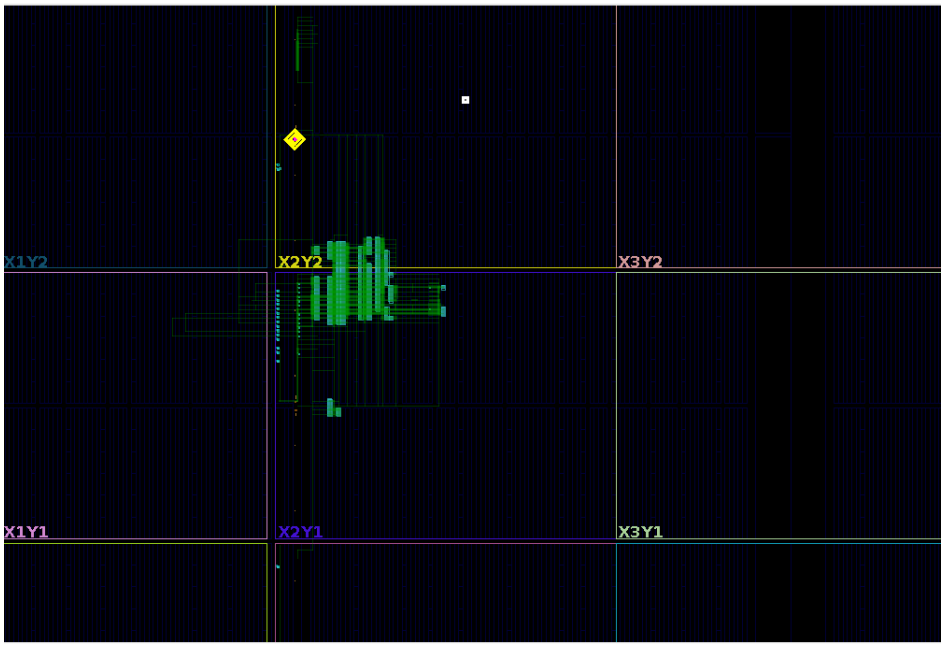



**図 2-20: クロック リージョン - KCU105**

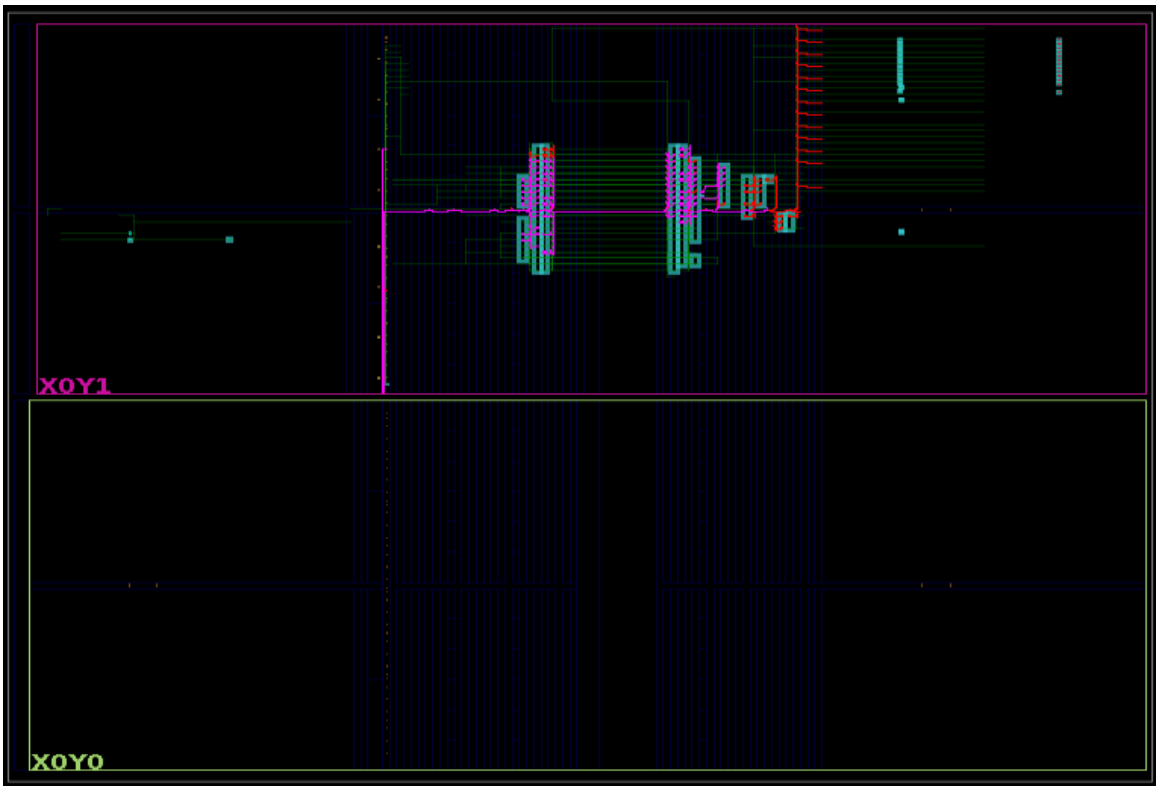



**図 2-21: クロック領域 X0Y1 および X0Y0 - SCU035**


© Copyright 2026 Advanced Micro Devices, Inc.

© Copyright 2026 Advanced Micro Devices, Inc.

### 4-5. [Device viewer] を使用して、各クロック領域で clk_rx (clk_out1) ドメインが使用しているクロック ルーティングおよび分配トラックを確認します。このドメインを駆動する BUFGCE から開始し、[Device viewer] で接続を解析します。

**4-5-1.** **[Schematic]** ビューで、**clk_gen_i0** セルの **clk_out1** ピン **(BUFGCE** の出力**)** に接続されている **clk_gen_i0** 内のネットを選択します。

ピンではなくネットを必ず選択してください。

**4-5-2.** **[Schematic]** ウィンドウで右クリックし、**[Highlight]** および識別しやすい色 **(**たとえば **[Color 18])** を選択して、選択したネットをハイライトします。

**4-5-3.** **clk_tx** に対して実行した解析を **clk_rx** に対しても繰り返します。

**4-5-4.** **clk_rx** ネットをハイライトしたままにしておきます。

#### 質問 10

clk_rx ドメインはどのクロック リージョンで使用されていますか?

---

---

---

### 4-6. [Device viewer] を使用して、各クロック領域で clk_samp ドメインによってどのクロック ルーティングおよびディストリビューション トラックが使用されているかを確認します。このドメインを駆動する BUFGCE から開始し、[Device viewer] で接続を解析します。

**4-6-1.** **[Schematic]** ビューが表示されていることを確認します。

**4-6-2.** **clk_gen_i0** 内の、**clk_gen_i0** セルの **CLK** ピン **(BUFGCE** の出力**)** に接続しているネットを選択します。

ピンではなく、必ずネットを選択してください。

**4-6-3.** **[Schematic]** ウィンドウ内を右クリックし、**[Highlight]** と識別しやすい色 **(**例**: [Color 19])** を選択して、選択したネットをハイライトします。

**4-6-4.** **clk_tx** に対して実行した解析を **clk_samp** に対しても繰り返します。


© Copyright 2026 Advanced Micro Devices, Inc.

**質問 11**

clk_samp ドメインはどのクロック領域で使用されていますか?

---

---

---

## 4-7. 重要: [Device] ビューアーでネットの表示を復元します。

**[Device] ビューアーでのネットの表示設定は、Vivado IDE の終了時にツールの環境設定の一部として自動的に保存されます。Vivado Design Suite を終了する前に、表示設定が元に戻っていることを確認してください。**

**4-7-1.** **[Show Cell Connections]** アイコン **()** をクリックします。

**4-7-2.** **[Show All]** をクリックします。

## 4-8. プロジェクトを閉じます。

**4-8-1.** **[File] > [Project] > [Close]** を選択してプロジェクトを閉じます。

[Close Project] ダイアログ ボックスが開きます。

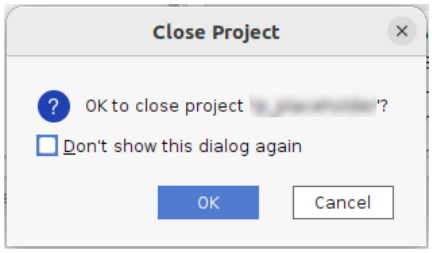



**図 2-22: Close Project ダイアログ ボックス**

**4-8-2.** **[OK]** をクリックします。

## 4-9. Vivado Design Suite を閉じます。

**4-9-1.** **[File] > [Exit]** を選択します。


© Copyright 2026 Advanced Micro Devices, Inc.

[Exit Vivado] ダイアログ ボックスが開きます。




**図 2-23: Exit Vivado ダイアログ ボックス**

**4-9-2.** プロジェクトまたはプロジェクトの一部を保存するかどうかを確認するメッセージが表示された場合は、保存するプロジェクトの要素を選択して **[Save]** をクリックし、選択した要素を保存します。保存しない場合は **[Don't Save]** をクリックします。

**4-9-3.** **Vivado Design Suite** を終了するか確認されたら、**[OK]** をクリックします。

**注記: Vivado Design Suite の終了時に確認が表示されないようにするため、[Don't show this dialog again] オプションを選択することもできます。**

一部のシステム (特に VM) では、メモリに制約がある場合があります。ワークスペースを削除するとディスク容量の一部が解放され、ほかのラボを実行できるようになります。

実行した演習が含まれるディレクトリは、グラフィカル インターフェイスまたはコマンドライン インターフェイスを使用して削除できます。いずれの方法を選択してもかまいません。どちらの手順でも、$TRAINING_PATH/clk_resources ディレクトリ内のすべてのファイルが再帰的に削除されます。

## 4-10. [オプション] [ローカル VM のみ—CloudShare 対象外] ファイル システムをクリーンアップします。

GUI の使用:

**4-10-1.** **$TRAINING_PATH/clk_resources** に移動します。

**4-10-2.** **[clk_resources]** を選択します。

**4-10-3.** **<Delete>** キーを押します。

-- または --


© Copyright 2026 Advanced Micro Devices, Inc.

**[Linux ユーザー]: コマンド ラインを使用する場合:**

**4-10-4.** **<Ctrl + Alt + T>** を押して **[terminal]** ウィンドウを開きます。

**4-10-5.** 次のコマンドを入力して、ワークスペースの内容を削除します。

```
[host] $ rm -rf $TRAINING_PATH/clk_resources
```

# まとめ

UltraScale™ および UltraScale+™ FPGA には、非常に柔軟なクロック管理コンポーネントが含まれています。MMCM、BUFG、および BUFCE を接続することで、複雑なクロック要件を持つシステムを実装できます。

[Clocking Wizard] は、MMCM を設定してグローバル バッファーに接続するのに役立つ便利なツールです。また、RTL デザイン内でグローバル クロック バッファーやリージョナル クロック バッファーをインスタンシエートし、手動で接続することもできます。

デザイン内のクロック ドメインの要件に基づいて、インプリメンテーション ツールは適切なクロック ルーティングおよびディストリビューション チャネルを選択してクロック ドメインをインプリメントします。セグメント化されたクロック アーキテクチャにより、デザイン内で膨大な数のクロック ドメインをサポートできるようになり、各ドメインは必要最小限のクロック リージョンおよびリソースのみを占有します。

# 解答

記載されている解答はサンプルのソリューションにすぎません。使用しているソフトウェア、Service Pack、またはオペレーティング システムのバージョンによって、結果が異なる場合があります。

1. なぜ [Clocking Wizard] を使用してコアを作成するのでしょうか。つまり、単に MMCM プリミティブをインスタンシエートするだけではいけないのでしょうか。

   [IP Catalog] の [Clocking Wizard] ツールは、MMCM 属性を設定するための使いやすいインターフェイスを提供します。さらに、生成されるコアには必要な入力バッファーやクロック バッファーをすべて含めることができます。これらはすべて、インスタンシエーション用の 1 つのコアにまとめられます。

2. clk_out2 に [No buffer] オプションを使用するのはなぜですか?


© Copyright 2026 Advanced Micro Devices, Inc.

clk_tx ドメインおよび clk_samp ドメインは、MMCME3_ADV の同じ出力から生成されますが、異なるバッファー (1 つはゲーテッド、もう 1 つはアンゲーテッド) を必要とします。

[Clocking Wizard] には、同一の出力を 2 つの異なるクロック バッファーでバッファリングする機能がないため、両方を [Clocking Wizard] で生成されたコアの外部でインスタンシエートする必要があります。

3. [Clocking Wizard] に表示されている入力クロック周波数、ならびに DIVCLK_DIVIDE および CLKFBOUT_MULT_F パラメータの値に基づくと、[Clocking Wizard] は VCO 周波数として何を選択していますか？

   VCO 周波数 FVCO は、次の式によって決定されます: FVCO = FIN * CLKFBOUT_MULT_F / DIVCLK_DIVIDE。

   したがって:

   **KCU105: FVCO = 300 MHz * 7.75 / 3 = 775 MHz**

   **SCU035: FVCO = 300 MHz * 15.5 / 3 = 1550 MHz**

4. [Clocking Wizard] に表示されている VCO 周波数と 2 つのクロックの [Divide] の値から、出力クロックは要求された周波数と一致しますか?

   出力 FOUT の周波数は、次の式によって決定されます: FOUT = FVCO / CLKOUT_DIVIDE。

   したがって:

   **KCU105:**

   FOUT1 = 775 MHz / 3.875 = 200 MHz

   FOUT2 = 775 MHz / 4 = 193.75 MHz

   **SCU035:**

   FOUT1 = 1550 MHz / 7.750 = 200 MHz

   FOUT2 = 1550 MHz /8 = 193.75 MHz

   両方の出力とも要求どおりです。

5. コアがすでに作成されている場合、そのコアはすでにデザインに接続されているのでしょうか。なぜこのデザインにクロッキング コアをインスタンシエートする必要があるのでしょうか。


© Copyright 2026 Advanced Micro Devices, Inc.

いいえ、コアの生成はコアを作成するだけであり、デザインに自動的に挿入されるわけではありません。そのため、デザイン内のモジュールにそのコアをインスタンス化し、適切なネットに接続する必要があります。

6. エディターは、ポートが有効なネットに接続されているかどうかをどのように示しますか？

   無効な接続は、テキストの背景が青色になって自動的にハイライト表示されます。ライン上にマウスを合わせると、エラーに関するポップアップ メッセージが表示されます。

7. 完成したデザインでは、MMCM およびグローバル クロック バッファー リソースがいくつ使用されましたか?

   **KCU105:**

   このデザインでは 1 つの MMCME3_ADV を使用しており、これは clk_core で使用されているものです。

   このデザインでは 4 つの BUFGCE (あらゆる種類の BUFG を実装するために使用されるプリミティブ) を使用します。2 つの BUFG は [Clocking Wizard] によって生成された BUFG で、1 つはクロック フィードバック用、もう 1 つは clk_rx 用です。残りの 2 つの BUFGCE は、手動でインスタンス化された clk_tx 用の BUFG と clk_samp 用の BUFGCE です。

   **SCU035:**

   デザインでは 1 つの MMCME4_ADV を使用しており、これは clk_core で使用されているものです。

   このデザインでは、4 個の BUFGCE (あらゆる種類の BUFG の実装に使用されるプリミティブ) を使用します。2 個の BUFG は [Clocking Wizard] によって生成された BUFG であり、1 つはクロック フィードバック用、もう 1 つは clk_rx 用です。残りの 2 個の BUFGCE は、手動でインスタンシエートされた clk_tx 用の BUFG と、clk_samp 用の BUFGCE です。

8. MMCM と 4 つのクロック バッファーはどのクロック領域に配置されていますか?

   **KCU105: MMCME3_ADV および 4 つの BUFGCE はすべてクロック領域 X2Y2 に配置されています。**

   **SCU035: MMCME4_ADV および 4 つの BUFGCE はすべてクロック領域 X0Y1 に配置されています。**


© Copyright 2026 Advanced Micro Devices, Inc.

9. clk_tx ドメインはどのクロック リージョンで使用されていますか？

   **KCU105: clk_tx クロック信号は、X2Y2 (生成元) と通過する X2Y1 の 2 つのクロック リージョンにまたがっています (これら 2 つのリージョンの両方で使用されます)。**

   **SCU035: clk_tx クロック信号は X0Y1 クロック領域にまたがっています。**

10. clk_rx ドメインは、どのクロック領域で使用されていますか?

    **KCU105: clk_rx は X2Y2 で生成され、X2Y1 で使用されます。**

    **SCU035: clk_rx は X0Y1 で使用されています。**

11. clk_samp ドメインはどのクロック リージョンで使用されていますか?

    **KCU105: clk_samp は、X2Y2 (生成) および X2Y1 (X2Y1 で使用) の 2 つのクロック リージョンにまたがっています。**

    **SCU035: clk_samp は X0Y1 クロック領域にまたがっています。**


© Copyright 2026 Advanced Micro Devices, Inc.